R Programming – Work example
https://www.educative.io/courses/learn-r-from-scratch/JPkVlp8wWVJ

Variables

Rule Variable in R

1. The variable name must start with letter and can contain number,letter,underscore(‘_’*) and period('.').*
2. ***Example:*** *variableName1,new.variable,*
   - *Underscore('_')* at the beginning of the variable name are not allowed. **Example:** '_my_var' is not a valid variable name.
   - Period('.') at the beginning of the variable name are allowed but should not be followed by a number. It is preferable in R to use '.' which helps to separate the different words for the identifier.
   - **Example:** '.myvar' is a valid variable name. However, '.1myvar' is not a valid variable name because the period followed by a number is not valid.
3. Reserved words or keywords are not allowed to be defined as a variable name.
4. Special characters such as '#', '&', etc., along with White space (tabs, space) are not allowed in a variable name.

name <- ojus
Error: object 'ojus' not found

String
name <-"ojus"
> name
[1] "ojus"

Numbers
> A <- 10
> A
[1] 10
> B<- 10.2
> B
[1] 10.2

Another way of printing output

name <- "Hello World"

```
> print(name)
[1] "Hello World"
> cat (name)
```

- **print ()** method prints its arguments on the R console.
- **cat()** does the same thing but is valid only for atomic types (logical, integer, real, complex, character) and names.
- We cannot call `cat()` on a **non-empty list** or any type of **object**.

R Script File

Writing an R program in a file and running it

```
# R program to add two numbers

numb1 <- 10
numb2 <- 20
sum <- 0
sum<- numb1 + numb2
sum
```

How to run? : ***Rscript add.r***

Concatenate Elements

You can also concatenate, or join, two or more elements, by using the paste() function.

To combine both text and a variable, R uses comma (,):

Eg.
```
text <- "awesome"
paste("R is", text)
```

Basic Data Types

Basic data types in R can be divided into the following types:

- numeric - (10.5, 55, 787)
- integer - (1L, 55L, 100L, where the letter "L" declares this as an integer)
- complex - (9 + 3i, where "i" is the imaginary part)
- character (a.k.a. string) - ("k", "R is exciting", "FALSE", "11.5")
- logical (a.k.a. boolean) - (TRUE or FALSE)

We can use the class() function to check the data type of a variable:

```
# numeric
x <- 10.5
class(x)

# integer
x <- 1000L
class(x)

# complex
x <- 9i + 3
class(x)

# character/string
x <- "R is exciting"
class(x)

# logical/boolean
x <- TRUE
class(x)
```

Type Conversion

You can convert from one type to another with the following functions:

- as.numeric()
- as.integer()
- as.complex()

String Length

There are many usesful string functions in R.

For example, to find the number of characters in a string, use the nchar() function:

Check a String

Use the grepl() function to check if a character or a sequence of characters are present in a string:

```r
str <- "Hello World!"

grepl("H", str)          TRUE
grepl("Hello", str)      TRUE
grepl("X", str)          FALSE
```

For Loops

A for loop is used for iterating over a sequence:

```r
for (x in 1:10) {
  print(x)
}
```

R Functions

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Creating a Function *Rscript functions.r*

To create a function, use the function() keyword:

```r
my_function <- function() {
  print("Hello World!")
}

my_function() # call the function named my_function
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

```
my_function <- function(x) {
  return (5 * x)
}
print(my_function(3))
print(my_function(5))
print(my_function(9))
```

Output : 15,25,45

Default Arguments

```
my_function <- function(x = 10) {
  return (5 * x)
}
print(my_function(3))
print(my_function())
print(my_function(9))
```

Output : 15,50,45

The most essential data structures used in R include:

- Vectors.
- Lists.
- Matrices.
- Dataframes.
- Factors.

Vectors *vectors.r*

A vector is simply a list of items that are of the same type.

To combine the list of items to a vector, use the c() function and separate the items by a comma.

In the example below, we create a vector variable called **fruits**, that combine strings:

```
# Vector of strings
fruits <- c("banana", "apple", "orange")

# Print fruits
fruits
```

To create a vector with values in a sequence, use the : operator:

```
# Vector with numerical values in a sequence

numbers <- 1:10

numbers

# Vector with numerical decimals in a sequence
numbers1 <- 1.5:6.5
numbers1
```

Lists

A list in R can contain many different data types inside it.

A list is a collection of data which is ordered and changeable.

To create a list, use the list() function:

```
# List of strings
thislist <- list("apple", "banana", "cherry",1,2)
# Print the list
thislist
```

Matrices

A matrix is a two dimensional data set with columns and rows.

A column is a vertical representation of data, while a row is a horizontal representation of data.

A matrix can be created with the matrix() function. Specify the nrow and ncol parameters to get the amount of rows and columns:

```
# Create a matrix
thismatrix <- matrix(c(1,2,3,4,5,6), nrow = 3, ncol = 2)

# Print the matrix
thismatrix
```

Access Matrix Items

You can access the items by using [ ] brackets. The first number "1" in the bracket specifies the row-position, while the second number "2" specifies the column-position:

```
thismatrix <- matrix(c("apple", "banana", "cherry", "orange"), nrow = 2, ncol = 2)
thismatrix[1, 2]
```

Access 2nd row

```
thismatrix <- matrix(c("apple", "banana", "cherry", "orange"), nrow = 2, ncol = 2)
thismatrix[2,]
```

Access 2nd Column

```
thismatrix <- matrix(c("apple", "banana", "cherry", "orange"), nrow = 2, ncol = 2)
thismatrix[,2]
```

Access More Than One Row

More than one row can be accessed if you use the c() function:

```
thismatrix <- matrix(c("apple", "banana", "cherry", "orange","grape", "pineapple", "pear", "melon", "fig"), nrow = 3, ncol = 3)
```

thismatrix[c(1,2),]

Adding Row To A Matrix (matrix-addrow.r)

We use function rbind() to add the row to any existing matrix.

Adding Column To A Matrix (matrix-addcol.r)

For adding a column to a Matrix in we use cbind() function.

R Data Frames

Data Frames

Data Frames are data displayed in a format as a table.

Data Frames can have different types of data inside it. While the first column can be character, the second and third can be numeric or logical. However, each column should have the same type of data.

Use the data.frame() function to create a data frame:

```
# Create a data frame
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)
# Print the data frame
Data_Frame
```

Summarize the Data

Use the summary() function to summarize the data from a Data Frame:

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
```

```
  Duration = c(60, 30, 45)
)
summary(Data_Frame)
```

```
Training      Pulse          Duration
 Other   :1   Min.   :100.0   Min.   :30.0
 Stamina :1   1st Qu.:110.0   1st Qu.:37.5
 Strength:1   Median :120.0   Median :45.0
              Mean   :123.3   Mean   :45.0
              3rd Qu.:135.0   3rd Qu.:52.5
              Max.   :150.0   Max.   :60.0
```

Add Rows *DF_rbind.r*

Use the rbind() function to add new rows in a Data Frame:

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)

# Add a new row
New_row_DF <- rbind(Data_Frame, c("Strength", 110, 110))

# Print the new row
New_row_DF
```

Number of Rows and Columns

Use the dim() function to find the amount of rows and columns in a Data Frame:

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)

dim(Data_Frame)
```

Factors

Factors are the data objects which are used to categorize the data and store it as levels. They are useful for storing categorical data. They can store both strings and integers. They are useful to categorize unique values in columns like “TRUE” or “FALSE”, or “MALE” or “FEMALE”, etc.. They are useful in data analysis for statistical modeling.

```
# R program to illustrate factors
# Creating factor using factor()
fac = factor(c("Male", "Female", "Male",
"Male", "Female", "Male", "Female"))
print(fac)
```

Output

```
Male   Female Male   Male   Female Male   Female
Levels: Female Male
```

How to install a package in R

Use

install.packages("package_name")

eg. install.packages("ggplot2")

R Plotting

Plot

The plot() function is used to draw points (markers) in a diagram.

The function takes parameters for specifying points in the diagram.

Parameter 1 specifies points on the **x-axis**.

Parameter 2 specifies points on the **y-axis**.

At its simplest, you can use the plot() function to plot two numbers against each other:

Example

Draw one point in the diagram, at position (1) and position (3):

```
plot(1, 3)
```

Multiple Points

You can plot as many points as you like, just make sure you have the same number of points in both axis:

Example

```
plot(c(1, 2, 3, 4, 5), c(3, 7, 8, 9, 12))
```

Sequences of Points

If you want to draw dots in a sequence, on both the **x-axis** and the **y-axis**, use the : operator:

Example

```
plot(1:10)
```

Draw a Line

The plot() function also takes a type parameter with the value l to draw a line to connect all the points in the diagram:

Example

```
plot(1:10, type="l")
```

Plot Labels

The plot() function also accept other parameters, such as main, xlab and ylab if you want to customize the graph with a main title and different labels for the x and y-axis:

Example

```
plot(1:10, main="My Graph", xlab="The x-axis", ylab="The y axis")
```

Graph Appearance

There are many other parameters you can use to change the appearance of the points.

Colors

Use col="*color*" to add a color to the points:

Example

```
plot(1:10, col="red")
```

Size

Use cex=*number* to change the size of the points (1 is default, while 0.5 means 50% smaller, and 2 means 100% larger):

Example

```
plot(1:10, cex=2)
```

Point Shape

Use pch with a value from 0 to 25 to change the point shape format:

Example

```
plot(1:10, pch=25, cex=2)
```

R Scatter Plot

Scatter Plots

A "scatter plot" is a type of plot used to display the relationship between two numerical variables, and plots one dot for each observation.

It needs two vectors of same length, one for the x-axis (horizontal) and one for the y-axis (vertical):

Example *scatter.r*

```
x <- c(5,7,8,7,2,2,9,4,11,12,9,6)
y <- c(99,86,87,88,111,103,87,94,78,77,85,86)
plot(x, y)
```

scatter1.r

```
x <- c(5,7,8,7,2,2,9,4,11,12,9,6)
y <- c(99,86,87,88,111,103,87,94,78,77,85,86)
plot(x, y, main="Observation of Cars", xlab="Car age", ylab="Car speed")
```

Box Plot

```
#draw in R terminal

x <- c(8, 5, 14, -9, 19, 12, 3, 9, 7, 4,

       4, 6, 8, 12, -8, 2, 0, -1, 5, 3)

boxplot(x, horizontal = TRUE)
```

Interpreting Box Plot

(https://www.open.edu/openlearn/mod/oucontent/view.php?printable =1&id =4089)

A box plot (aka box and whisker plot) uses boxes and lines to depict the distributions of one or more groups of numeric data. Box limits indicate the range of the central 50% of the data, with a central line marking the median value.

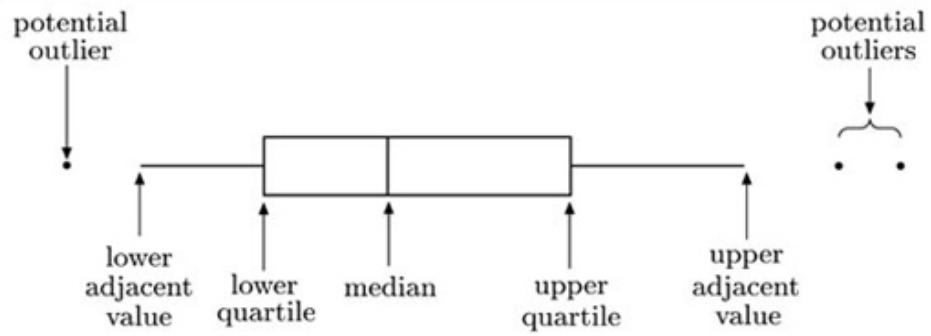
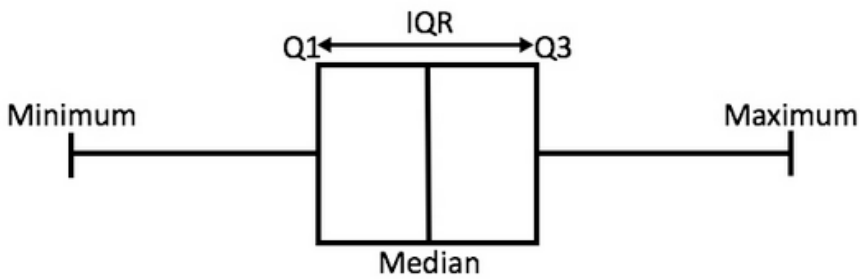
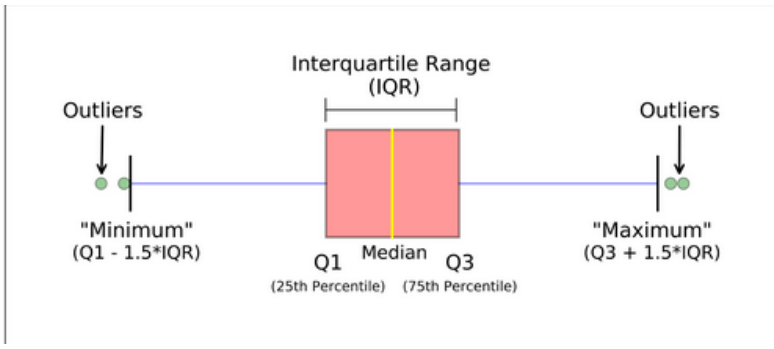


Figure 1.8 A typical boxplot



The interquartile range (IQR) is the box plot showing the middle 50% of



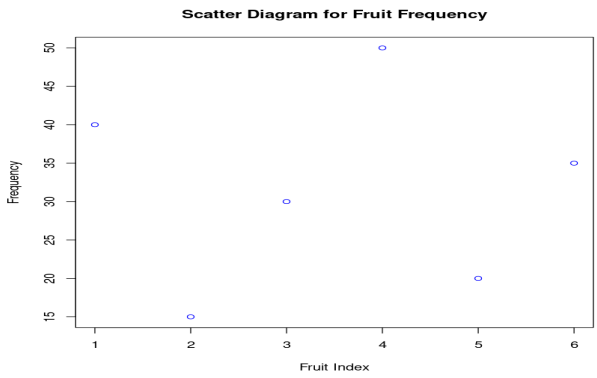
The ends of the box mark the quartiles, and the vertical line through the box is located at the median.

The **whiskers** of a boxplot extend to values known as adjacent values. These are the values in the data that are furthest away from the median on either side of the box, but are still within a distance of 1.5 times the interquartile range from the nearest end of the box (that is, the nearer quartile). In many cases the whiskers actually extend right out to the most extreme values in the data set. However, in other cases they do not.

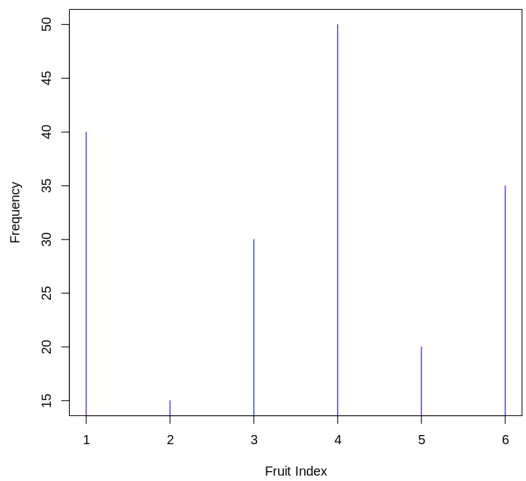
Any values in the data set that are more extreme than the adjacent values are plotted as separate points on the boxplot. This identifies them as potential outliers that may need further investigation.

When a data distribution is symmetric, you can expect the median to be in the exact center of the box: the distance between Q1 and Q2 should be the same as between Q2 and Q3. Outliers should be evenly present on either side of the box. If a distribution is skewed, then the median will not be in the middle of the box, and instead off to the side. You may also find an imbalance in the whisker lengths, where one side is short with no outliers, and the other has a long tail with many more outliers.

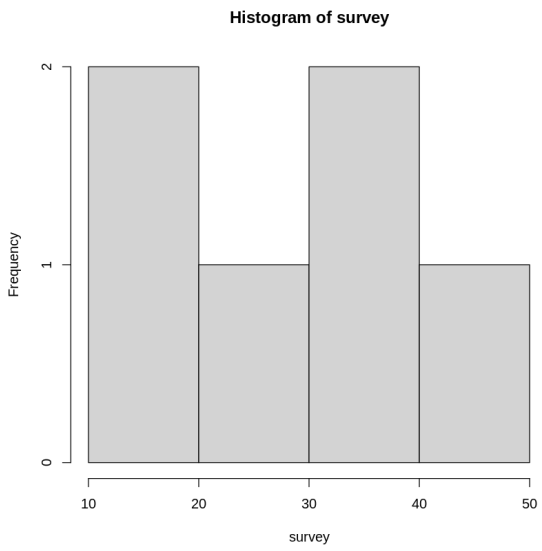
```
survey <- c(apple=40, kiwi=15, grape=30, banana=50, pear=20, orange=35)

#To find relation betweenfruit index and frequency.

plot(survey,xlab = "Fruit Index", ylab = "Frequency", col="blue", main =
"Scatter Diagram for Fruit Frequency")
```



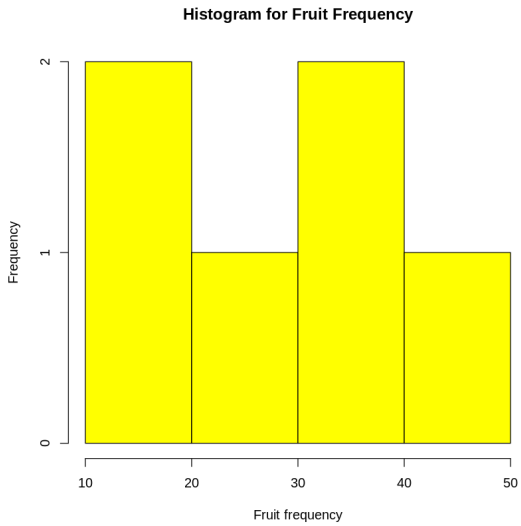

```
plot(survey, xlab = "Fruit Index", ylab = "Frequency", type = "h", col =
"blue")
```




```
hist(survey)
```




```
hist(survey,xlab = "Fruit frequency", main = "Histogram for Fruit
Frequency", col = "yellow")
```



h

ttp://www.sthda.com/english/wiki/r-base-graphs

Line graphs

```
plot(x, y, type = "l", lty = 1)
lines(x, y, type = "l", lty = 1)
```

The function **lines**() can not produce a plot on its own.

However, it can be used to add **lines**() on an existing graph.

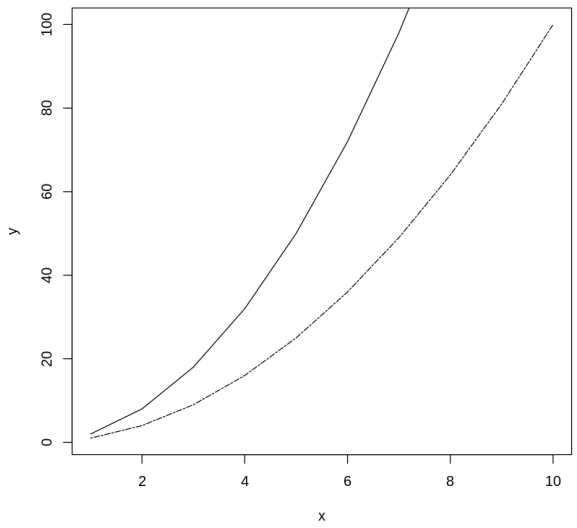
This means that, first you have to use the function **plot**() to create an empty graph and then use the function **lines**() to add lines.

x, y: coordinate vectors of points to join

- **type**: character indicating the type of plotting. Allowed values are:
  - "p" for points

    - “l” for lines
    - “b” for both points and lines
    - “c” for empty points joined by lines
    - “o” for overplotted points and lines
    - “s” and “S” for stair steps
    - “n” does not produce any points or lines
- **lty**: line types. Line types can either be specified as an integer (0=blank, 1=solid (default), 2=dashed, 3=dotted, 4=dotdash, 5=longdash, 6=twodash) or as one of the character strings “blank”, “solid”, “dashed”, “dotted”, “dotdash”, “longdash”, or “twodash”, where “blank” uses ‘invisible lines’ (i.e., does not draw them).

```
x<-c(1:10)
y<- x*x
x1<-c(2:10)
y2 <- y*2
plot(x, y, type = "l", lty = 6)
lines(x, y2, type = "l", lty = 1)
```



Introduction to ggplot2

(http://www.sthda.com/english/wiki/qplot-quick-plot-with-ggplot2-r-software-and-data-visualization)

Install and load ggplot2 package

```
# Installation
install.packages('ggplot2')
# Loading
library(ggplot2)
```

The data set **mtcars** is used in the examples below:

```
# Load the data
data(mtcars)
df <- mtcars[, c("mpg", "cyl", "wt")]
head(df)
```

- The function **qplot()** [in **ggplot2**] is very similar to the basic **plot()** function from the **R** base package.
- It can be used to create and combine easily different types of plots.
- However, it remains less flexible than the function **ggplot()**.

A simplified format of **qplot()** is :

```
qplot(x, y=NULL, data, geom="auto",
      xlim = c(NA, NA), ylim =c(NA, NA))
```

- **x** : x values
- **y** : y values (optional)
- **data** : data frame to use (optional).
- **geom** : Character vector specifying geom to use. Defaults to “point” if x and y are specified, and “histogram” if only x is specified.
- **xlim**, **ylim**: x and y axis limits

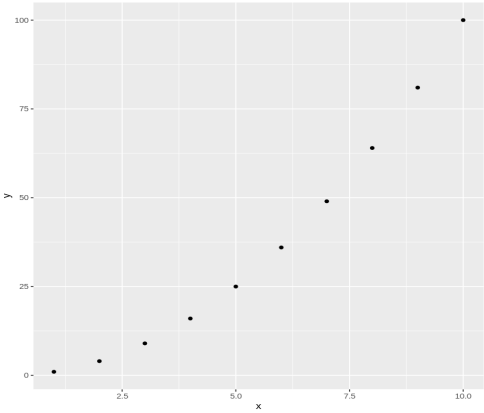
Other arguments including *main*, *xlab*, *ylab* and *log* can be used also:

- **main**: Plot title
- **xlab, ylab**: x and y axis labels
- **log**: which variables to log transform. Allowed values are “x”, “y” or “xy”

Scatter plots

```
# Use data from numeric vectors
```

```
x <- 1:10; y = x*x
# Basic plot
qplot(x,y)
# Add line
qplot(x, y, geom=c("point", "line"))
# Use data from a data frame
qplot(mpg, wt, data=mtcars)
```



Hexbinplot for Large Datasets

- Although color and transparency can be used in a scatterplot to address this issue, a hexbinplot is sometimes a better alternative.
- A hexbinplot combines the ideas of scatterplot and histogram. Similar to a scatterplot, a hexbinplot visualizes data in the x- axis and y-axis.
- Data is placed into hexbins, and the third dimension uses shading to
- represent the concentration of data in each hexbin.
-

Data Exploration Versus Presentation

- Using visualization for data exploration is different from presenting results to stakeholders.
- Not every type of plot is suitable for all audiences.
-
- Most of the plots presented earlier try to detail the data as clearly as possible for data scientists to identify structures and relationships.
- These graphs are more technical in nature and are better
- suited to technical audiences such as data scientists.

- Nontechnical stakeholders generally prefer simple, clear graphics that focus on the message rather than the data.
-

eg. Density plots, bar plots etc

Data Import and Export in R

(https://rpubs.com/liamroel13/mod2_les4)

R base functions for importing data

The R base function read.table() is a general function that can be used to read a file in table format. The data will be imported as a data frame.

Note that, depending on the format of your file, several variants of read.table() are available to make your life easier, including read.csv(), read.csv2(), read.delim() and read.delim2().

```
# Read tabular data into R
read.table(file, header = FALSE, sep = "", dec = ".")
# Read "comma separated value" files (".csv")
read.csv(file, header = TRUE, sep = ",", dec = ".", ...)
# Or use read.csv2: variant used in countries that
# use a comma as decimal point and a semicolon as field separator.
read.csv2(file, header = TRUE, sep = ";", dec = ",", ...)
# Read TAB delimited files
read.delim(file, header = TRUE, sep = "\t", dec = ".", ...)
read.delim2(file, header = TRUE, sep = "\t", dec = ",", ...)
```

- **file**: the path to the file containing the data to be imported into R.
- **sep**: the field separator character. “ is used for tab-delimited file.
- **header**: logical value. If TRUE, read.table() assumes that your file has a header row, so row 1 is the name of each column. If that’s not the case, you can add the argument header = FALSE.
- **dec**: the character used in the file for decimal points.

Eg : Import a CSV file ‘sales.csv’ to a data frame ‘sales’.

```
sales <- read.csv(file = 'sales.csv')

head(sales)
```

The **head() and tail()** function in R are often used to read the first and last n rows of a dataset. (https://www.journaldev.com/43863/head-and-tail-function-r)

Syntax of head() and tail() function

head(x, n=number) # default number n is 6

```
#importing the dataset
df<-datasets::iris

#returns first n rows of the data
head(df)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
1          5.1         3.5          1.4         0.2     setosa
2          4.9         3.0          1.4         0.2     setosa
3          4.7         3.2          1.3         0.2     setosa
4          4.6         3.1          1.5         0.2     setosa
5          5.0         3.6          1.4         0.2     setosa
6          5.4         3.9          1.7         0.4     setosa
```

if head(df,10) is given it will display 10 rows instead of default 6

tail(x,n=number) # default number n is 6

```
#importing the data
df<-datasets::airquality

#returns first 10 rows
head(df,n=10)
```

```
   Ozone Solar.R Wind Temp Month Day
1     41     190  7.4   67     5   1
2     36     118  8.0   72     5   2
3     12     149 12.6   74     5   3
4     18     313 11.5   62     5   4
5     NA      NA 14.3   56     5   5
6     28      NA 14.9   66     5   6
7     23     299  8.6   65     5   7
8     19      99 13.8   59     5   8
9      8      19 20.1   61     5   9
10    NA     194  8.6   69     5  10
```

Examine a Data Frame in R with 7 Basic Functions (DataFrameStructure.r)

1 dim(): shows the dimensions of the data frame by row and column
2 str(): shows the structure of the data frame
3 summary(): provides summary statistics on the columns of the data frame
4 colnames(): shows the name of each column in the data frame
5 head(): shows the first 6 rows of the data frame
6 tail(): shows the last 6 rows of the data frame
7 View(): shows a spreadsheet-like display of the entire data frame

Reading a file from internet

It's possible to use the functions read.delim(), read.csv() and read.table() to import files from the web.

```
my_data <- read.delim("http://www.sthda.com/upload/boxplot_format.txt")
head(my_data)
```

Writing Data from R to txt

You can use the **base** functions or the functions from the **readr** package to write data from R to txt.

- R **base** functions for writing data: write.table(), write.csv(), write.csv2()
- **readr** functions for writing data: write_tsv(), write_csv()

1.) Use the R base functions

```
 #Loading mtcars data
data("mtcars")
# Write data to txt file: tab separated values
# sep = "\t"
write.table(mtcars, file = "mtcars.txt", sep = "\t",
            row.names = TRUE, col.names = NA)
# Write data to csv files:
# decimal point = "." and value separators = comma (",")
write.csv(mtcars, file = "mtcars.csv")
# Write data to csv files:
# decimal point = comma (",") and value separators = semicolon (";")
write.csv2(mtcars, file = "mtcars.csv")
```

Reading XML File

The xml file is read by R using the function **xmlParse()**. It is stored as a list in R.

```
# Load the package required to read XML files.
library("XML")

# Also load the other required package.
library("methods")

# Give the input file name to the function.
result <- xmlParse(file = "input.xml")

# Print the result.
print(result)
```

XML to Data Frame

To handle the data effectively in large files we read the data in the xml file as a data frame. Then process the data frame for data analysis.

```
# Load the packages required to read XML files.
```

```
library("XML")
library("methods")

# Convert the input xml file to a data frame.
xmldataframe <- xmlToDataFrame("input.xml")
print(xmldataframe)
```

Read the JSON File

(*https://www.tutorialspoint.com/r/r_json_files.htm*)

```
{
  "ID":["1","2","3","4","5","6","7","8" ],
  "Name":["Rick","Dan","Michelle","Ryan","Gary","Nina","Simon","Guru" ],
  "Salary":["623.3","515.2","611","729","843.25","578","632.8","722.5" ],

  "StartDate":[ "1/1/2012","9/23/2013","11/15/2014","5/11/2014","3/27/2015","5/21/2013",
    "7/30/2013","6/17/2014"],
  "Dept":[ "IT","Operations","IT","HR","Finance","IT","Operations","Finance"]
}
```

The JSON file is read by R using the function from **JSON()**. It is stored as a list in R.

```
# Load the package required to read JSON files.
library("rjson")

# Give the input file name to the function.
result <- fromJSON(file = "input.json")

# Print the result.
print(result)
```

Convert JSON to a Data Frame

We can convert the extracted data above to a R data frame for further analysis using the **as.data.frame()** function.

```
# Load the package required to read JSON files.

library("rjson")

# Give the input file name to the function.

result <- fromJSON(file = "input.json")



# Convert JSON file to a data frame.
```

```
json_data_frame <- as.data.frame(result)

print(json_data_frame)
```

R - Mean, Median and Mode

Mean

It is calculated by taking the sum of the values and dividing with the number of values in a data series.

The function **mean()** is used to calculate this in R.

Syntax

The basic syntax for calculating mean in R is −

```
mean(x, trim = 0, na.rm = FALSE, ...)
```

Following is the description of the parameters used −

- **x** is the input vector.
- **trim** is used to drop some observations from both end of the sorted vector.
- **na.rm** is used to remove the missing values from the input vector.

```
# Create a vector.
x <- c(1,2,3,4,5)

# Find Mean.
result.mean <- mean(x)
print(result.mean)
```

Applying NA Option

If there are missing values, then the mean function returns NA.

To drop the missing values from the calculation use na.rm = TRUE. which means remove the NA values.

```
# Create a vector.

x <- c(12,7,3,4.2,18,2,54,-21,8,-5,NA)

# Find mean.
result.mean <-  mean(x)
```

```
print(result.mean)

# Find mean dropping NA values.
result.mean <-  mean(x,na.rm = TRUE)
print(result.mean)
```

Median

The middle most value in a data series is called the median. The **median()** function is used in R to calculate this value.

Syntax

The basic syntax for calculating median in R is −

```
median(x, na.rm = FALSE)
```

Following is the description of the parameters used −

- **x** is the input vector.
- **na.rm** is used to remove the missing values from the input vector.

```
# Create the vector.

x <- c(12,7,3,4.2,18,2,54,-21,8,-5)

# Find the median.
median.result <- median(x)
print(median.result)
```

Mode

The mode is the value that has highest number of occurrences in a set of data. Unike mean and median, mode can have both numeric and character data.

```
# Create the function.

getmode <- function(v) {
   uniqv <- unique(v)
   uniqv[which.max(tabulate(match(v, uniqv)))]
}

# Create the vector with numbers.
v <- c(2,1,2,3,1,2,3,4,1,5,5,3,2,3)

# Calculate the mode using the user function.
result <- getmode(v)
print(result)
```

```
# Create the vector with characters.
charv <- c("o","it","the","it","it")

# Calculate the mode using the user function.
result <- getmode(charv)
print(result)
```

Descriptive Statistics (*https://statsandr.com/blog/descriptive-statistics-in-r/*)

Descriptive Statistics

Summary() - summary data

cov(x,y) - returns covariance

What is covariance?

Covariance signifies the direction of the linear relationship between the two variables. By direction we mean if the variables are directly proportional or inversely proportional to each other.

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x}) * (y_i - \bar{y})}{N}$$

cor(x,y) - returns
correlation

What is correlation?

Correlation is a statistical measure that expresses the extent to which two variables are linearly related (meaning they change together at a constant rate). It's a common tool for describing simple relationships without making a statement about cause and effect.

$$Correlation = \frac{Cov\ (x, y)}{\sigma x * \sigma y}$$

IQR(x) – returns
Interquartile range

mean(x) - returns mean

median(x) - returns median

range(x) - returns min max

sd(x) - returns std. dev.

var(x) - returns variance

```
dat <- iris # load the iris dataset and renamed it dat
```

```
head(dat) # first 6 observations
```

```
str(dat) # structure of dataset
```

Minimum and maximum

Minimum and maximum can be found thanks to the min() and max() functions:

```
min(dat$Sepal.Length)
```

```
## [1] 4.3
```

```
max(dat$Sepal.Length)
```

```
## [1] 7.9
```

Alternatively the range() function: list max and minimum

```
rng <- range(dat$Sepal.Length)
rng
```

```
## [1] 4.3 7.9
```

First and third quartile

As the median, the first and third quartiles can be computed thanks to the quantile() function and by setting the second argument to 0.25 or 0.75:

```
quantile(dat$Sepal.Length, 0.25) # first quartile
```

```
## 25%
## 5.1
```

```
quantile(dat$Sepal.Length, 0.75) # third quartile
```

```
## 75%
## 6.4
```

Interquartile range

The interquartile range (i.e., the difference between the first and third quartile) can be computed with the IQR() function:

```
IQR(dat$Sepal.Length)
```

```
## [1] 1.3
```

or alternatively with the quantile() function again:

```
quantile(dat$Sepal.Length, 0.75) - quantile(dat$Sepal.Length, 0.25)
```

Standard deviation and variance

The standard deviation and the variance is computed with the sd() and var() functions:

```
sd(dat$Sepal.Length) # standard deviation
```

```
## [1] 0.8280661
```

```
var(dat$Sepal.Length) # variance
```

```
## [1] 0.6856935
```

to compute the standard deviation (or variance) of multiple variables at the same time, use lapply() with the appropriate statistics as second argument:

```
lapply(dat[, 1:4], sd)
```

```
## $Sepal.Length
## [1] 0.8280661
##
## $Sepal.Width
## [1] 0.4358663
##
## $Petal.Length
## [1] 1.765298
##
## $Petal.Width
## [1] 0.7622377
```

Summary

You can compute the minimum, 1st

quartile, median, mean, 3rd

quartile and the maximum for all numeric variables of a dataset at once using summary():

```
summary(dat)
```

```
##   Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
```

```
##  Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100
##  1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
##  Median :5.800   Median :3.000   Median :4.350   Median :1.300
##  Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
##  3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
##  Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500
##        Species
##  setosa     :50
##  versicolor:50
##  virginica :50
##
##
##
```

if you need these descriptive statistics by group use the by() function:

```
by(dat, dat$Species, summary)
```

```
## dat$Species: setosa
##   Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
##  Min.   :4.300   Min.   :2.300   Min.   :1.000   Min.   :0.100
##  1st Qu.:4.800   1st Qu.:3.200   1st Qu.:1.400   1st Qu.:0.200
##  Median :5.000   Median :3.400   Median :1.500   Median :0.200
##  Mean   :5.006   Mean   :3.428   Mean   :1.462   Mean   :0.246
##  3rd Qu.:5.200   3rd Qu.:3.675   3rd Qu.:1.575   3rd Qu.:0.300
##  Max.   :5.800   Max.   :4.400   Max.   :1.900   Max.   :0.600
##        Species
##  setosa     :50
##  versicolor: 0
##  virginica : 0
##
##
##
## ------------------------------------------------------------
## dat$Species: versicolor
##   Sepal.Length    Sepal.Width     Petal.Length   Petal.Width          Species
##  Min.   :4.900   Min.   :2.000   Min.   :3.00   Min.   :1.000   setosa     : 0
##  1st Qu.:5.600   1st Qu.:2.525   1st Qu.:4.00   1st Qu.:1.200   versicolor:50
##  Median :5.900   Median :2.800   Median :4.35   Median :1.300   virginica : 0
##  Mean   :5.936   Mean   :2.770   Mean   :4.26   Mean   :1.326
##  3rd Qu.:6.300   3rd Qu.:3.000   3rd Qu.:4.60   3rd Qu.:1.500
##  Max.   :7.000   Max.   :3.400   Max.   :5.10   Max.   :1.800
## ------------------------------------------------------------
## dat$Species: virginica
##   Sepal.Length    Sepal.Width     Petal.Length    Petal.Width
##  Min.   :4.900   Min.   :2.200   Min.   :4.500   Min.   :1.400
##  1st Qu.:6.225   1st Qu.:2.800   1st Qu.:5.100   1st Qu.:1.800
##  Median :6.500   Median :3.000   Median :5.550   Median :2.000
```

```
##  Mean   :6.588   Mean   :2.974   Mean   :5.552   Mean   :2.026
##  3rd Qu.:6.900   3rd Qu.:3.175   3rd Qu.:5.875   3rd Qu.:2.300
##  Max.   :7.900   Max.   :3.800   Max.   :6.900   Max.   :2.500
##        Species
##  setosa     : 0
##  versicolor: 0
##  virginica :50
##
##
##
```

Exploratory Data Analysis in R

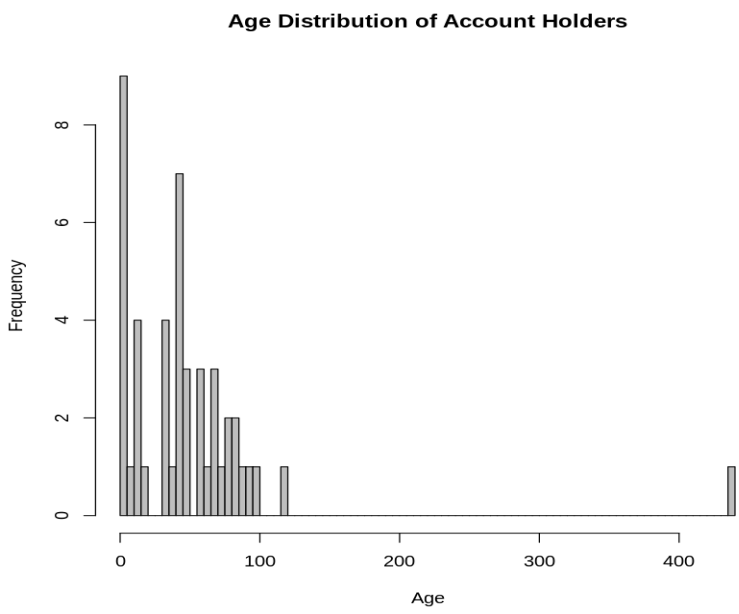
(https://towardsdatascience.com/exploratory-data-analysis-in-r-for-beginners-fe031add7072)

Exploratory Data Analysis (**EDA**) is the process of analyzing and visualizing the data to get a better understanding of the data and glean insight from it. There are various steps involved when doing EDA but the following are the common steps that a data analyst can take when performing EDA:

1. Import data
2. Clean the data
3. Process the data
4. Visualize the data

Visualization before Analysis

```
age <- c(0,0,0,0,0,0,0,0,0,10,12,13,14,15,16,34, 35,34,36,440,42,44,45,46,47,34,45,43,42,43,46,56,57,
    60,67,68,63,70,75,78,80,82,84,90,95,100,120)
hist(age, breaks=100,  main='Age Distribution of Account Holders',
xlab='Age', ylab='Frequency', col='gray')
```



Handling Missing data

```
x <- c(1, 2, 3, NA, 4)

is.na(x)

mean(x)

[1] NA

mean(x, na.rm=TRUE)

[1] 2.5
```

The na.exclude() function returns the object with incomplete cases removed.

```
DF <- data.frame(x = c(1, 2, 3), y = c(10, 20, NA))

DF

x y

1 1 10

2 2 20

3 3 NA

DF1 <- na.exclude(DF)

DF1

x y

1 1 10

2 2 20
```

Visualizing a Single Variable

Table 3.4 Example Functions for Visualizing a Single Variable

Function	Purpose
`plot(data)`	Scatterplot where x is the index and y is the value; suitable for low-volume data
`barplot(data)`	Barplot with vertical or horizontal bars
`dotchart(data)`	Cleveland dot plot [12]
`hist(data)`	Histogram
`plot(density(data))`	Density plot (a continuous histogram)
`stem(data)`	Stem-and-leaf plot
`rug(data)`	Add a rug representation (1-d plot) of the data to an existing plot

Dot charts and bar plots

- Dotchart and barplot portray continuous values with labels from a discrete variable.
- A dotchart can be created in R with the function dotchart(x, label=...) , where x is a numeric vector and label is a vector of categorical labels for x .
- A barplot can be created with the barplot(height) function, where height represents a vector or matrix.

```
data(mtcars)

dotchart(mtcars$mpg,labels=row.names(mtcars),cex=.7,

main="Miles Per Gallon (MPG) of Car Models", xlab="MPG" )

barplot(table(mtcars$cyl), main="Distribution of Car Cylinder Counts",

        xlab="Number of Cylinders")
```

Histogram and Density Plot

```
# randomly generate 4000 observations from the log normal distribution

income <- rlnorm(4000, meanlog = 4, sdlog = 0.7)
```

```
summary(income)

    Min. 1st Qu. Median Mean 3rd Qu. Max.

    4.301 33.720 54.970 70.320 88.800 659.800

income <- 1000*income

summary(income)

    Min. 1st Qu. Median Mean 3rd Qu. Max.

    4301 33720 54970 70320 88800 659800

# plot the histogram

hist(income, breaks=500, xlab="Income", main="Histogram of Income")

# density plot

plot(density(log10(income), adjust=0.5),

main="Distribution of Income (log10 scale)")
```

Basic statistical tests Using R

(**https://www.dataanalytics.org.uk/basic-statistical-tests-using-r/**)

R can carry out a wide range of statistical analyses. Some of the simpler ones include:

- Summary statistics (e.g. mean, standard deviation).

- Two-sample differences tests (e.g. t-test).
- Non-parametric tests (e.g. U-test).
- Matched pairs tests (e.g. Wilcoxon).
- Association tests (e.g. Chi squared).
- Goodness of Fit tests.

These are the subject of this section.

Basic statistics

Really simple summary stats

R has a range of functions for carrying out summary statistics. The following table shows a few of the functions that operate on single variables.

Statistic	**Function**
Sum	sum(x)
Mean	mean(x)
Median	median(x)
Largest	max(x)
Smallest	min(x)
Standard Deviation	sd(x)
Variance	var(x)
Number of elements	length(x)

T-test (https://www.javatpoint.com/t-test-in-r)

- In statistics, the T-test is one of the most common test which is used to determine whether the mean of the two groups is equal to each other.
- The assumption for the test is that both groups are sampled from a normal distribution with equal fluctuation.
- The null hypothesis is that the two means are the same, and the alternative is that they are not identical.
- Such samples are described as being parametric and the t-test is a parametric test.

- For evaluating the statistical significance of the **t-test**, we need to compute the **p-value**. The p-value range starts from 0 to 1, and is interpreted as follow:

- If the p-value is lower than 0.05, it means we are strongly confident to reject the null hypothesis(H0). So that alternate hypothesis(H1) is accepted.
- If the p-value is higher than 0.05, then it indicates that we don't have enough evidence to reject the null hypothesis.

- In R the t.test() command will carry out several versions of the t-test.

```
t.test(x, y, alternative, mu, paired, var.equal, …)
```

- x – a numeric sample.
- y – a second numeric sample (if this is missing the command carries out a 1-sample test).
- alternative – how to compare means, the default is “two.sided”. You can also specify “less” or “greater”.
- mu – the true value of the mean (or mean difference). The default is 0.
- paired – the default is paired = FALSE. This assumes independent samples. The alternative paired = TRUE is used for matched pair tests.
- equal – the default is var.equal = FALSE. This treats the variance of the two samples separately. If you set var.equal = TRUE you conduct a classic t-test using pooled variance.
- … – there are additional parameters that we aren’t concerned with here.

In most cases you want to compare two independent samples:

```
> X
[1] 12 15 17 11 15
> Y
[1] 8 9 7 9

> t.test(X, Y)

Welch Two Sample t-test

data:  X and Y
t = 4.8098, df = 5.4106, p-value = 0.003927
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
2.745758 8.754242
sample estimates:
mean of x mean of y
14.00       8.25
```

If you specify a single variable you can carry out a 1-sample test by specifying the mean to compare against:

```
> data1
[1] 3 5 7 5 3 2 6 8 5 6 9
> t.test(data1, mu = 5)

One Sample t-test

data:  data1
t = 0.55902, df = 10, p-value = 0.5884
alternative hypothesis: true mean is not equal to 5
95 percent confidence interval:
3.914249 6.813024
sample estimates:
mean of x
5.363636
```

If you have matched pair data you can specify paired = TRUE as a parameter.

U-test (http://www.sthda.com/english/wiki/unpaired-two-samples-wilcoxon-test-in-r)

- The U-test is used for comparing the median values of two samples.
- You use it when the data are not normally distributed, so it is described as a non-parametric test.
- The U-test is often called the Mann-Whitney U-test but is generally attributed to Wilcoxon (Wilcoxon Rank Sum test),
- If the p-value evaluated is less than significance level alpha, then null hypothesis is rejected.

R the command is wilcox.test().

wilcox.test(x, y, alternative, mu, paired, …)

- x – a numeric sample.
- y – a second numeric sample (if this is missing the command carries out a 1-sample test).
- alternative – how to compare means, the default is "two.sided". You can also specify "less" or "greater".
- mu – the true value of the median (or median difference). The default is 0.
- paired – the default is paired = FALSE. This assumes independent samples. The alternative paired = TRUE is used for matched pair tests.

- … – there are additional parameters that we aren't concerned with here.

```
> # Data in two numeric vectors
women_weight <- c(38.9, 61.2, 73.3, 21.8, 63.4, 64.6, 48.4, 48.8, 48.5)
men_weight <- c(67.8, 60, 63.4, 76, 89.4, 73.3, 67.3, 61.3, 62.4)
# Create a data frame
my_data <- data.frame(
              group = rep(c("Woman", "Man"), each = 9),
              weight = c(women_weight,  men_weight)
              )
 res <- wilcox.test(women_weight, men_weight, exact=FALSE)
 res

Wilcoxon rank sum test with continuity correction
data:  weight by group
W = 66, p-value = 0.02712
alternative hypothesis: true location shift is not equal to 0
```

The **p-value** of the test is 0.02712, which is less than the significance level alpha = 0.05. We can conclude that men's median weight is significantly different from women's median weight with a **p-value** = 0.02712.

Paired tests (http://www.sthda.com/english/wiki/paired-samples-t-test-in-r)

- The t-test and the U-test can both be used when your data are in matched pairs.
- Sometimes this kind of test is also called a repeated measures test (depending on circumstance). You can run the test by adding paired = TRUE to the appropriate command.
- The **p-value** of the test is less than the significance level alpha = 0.05. We can then reject null hypothesis
- Here is an example where the data show the effectiveness of some treatement on mice

```
Weight of the mice before treatment
before <-c(200.1, 190.9, 192.7, 213, 241.4, 196.9, 172.2, 185.5, 205.2, 193.7)
# Weight of the mice after treatment
after <-c(392.9, 393.2, 345.1, 393, 434, 427.9, 422, 383.9, 392.3, 352.2)
# Create a data frame
my_data <- data.frame(
              group = rep(c("before", "after"), each = 10),
              weight = c(before,  after)
              )
 # Compute paired t-test
res <- t.test(before, after, paired = TRUE)
res
Paired t-test
```

```
data:  before and after
t = -20.883, df = 9, p-value = 6.2e-09
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -215.5581 -173.4219
sample estimates:
mean of the differences
               -194.49
```

You can do a similar thing with the wilcox.test().

res <- wilcox.test(weight ~ group, data = my_data, paired = TRUE)

res

Wilcoxon signed rank exact test

data: weight by group

V = 55, p-value = 0.001953

alternative hypothesis: true location shift is not equal to 0

Chi Squared tests (https://www.geeksforgeeks.org/chi-square-test-in-r/)

The chi-square test of independence evaluates whether there is an association between the categories of the two variables.

We will take the survey data in the **MASS** library which represents the data from a survey conducted on students.

Our aim is to test the hypothesis whether the students smoking habit is independent of their exercise level at .05 significance level.

R program to illustrate

Chi-Square Test in R

library(MASS)

print(str(survey))

stu_data = data.frame(survey$Smoke,survey$Exer)

stu_data = table(survey$Smoke,survey$Exer)

print(stu_data)

print(chisq.test(stu_data))

```
'data.frame': 237 obs. of  12 variables:
 $ Sex   : Factor w/ 2 levels "Female","Male": 1 2 2 2 2 1 2 1 2 2 ...
 $ Wr.Hnd: num  18.5 19.5 18 18.8 20 18 17.7 17 20 18.5 ...
 $ NW.Hnd: num  18 20.5 13.3 18.9 20 17.7 17.7 17.3 19.5 18.5 ...
 $ W.Hnd : Factor w/ 2 levels "Left","Right": 2 1 2 2 2 2 2 2 2 2 ...
 $ Fold  : Factor w/ 3 levels "L on R","Neither",..: 3 3 1 3 2 1 1 3 3 3 ...
 $ Pulse : int  92 104 87 NA 35 64 83 74 72 90 ...
 $ Clap  : Factor w/ 3 levels "Left","Neither",..: 1 1 2 2 3 3 3 3 3 3 ...
 $ Exer  : Factor w/ 3 levels "Freq","None",..: 3 2 2 2 3 3 1 1 3 3 ...
 $ Smoke : Factor w/ 4 levels "Heavy","Never",..: 2 4 3 2 2 2 2 2 2 2 ...
 $ Height: num  173 178 NA 160 165 ...
 $ M.I   : Factor w/ 2 levels "Imperial","Metric": 2 1 NA 2 2 1 1 2 2 2 ...
 $ Age   : num  18.2 17.6 16.9 20.3 23.7 ...
NULL

        Freq None Some
  Heavy    7    1    3
  Never   87   18   84
  Occas   12    3    4
  Regul    9    1    7

        Pearson's Chi-squared test

data:  stu_data
X-squared = 5.4885, df = 6, p-value = 0.4828
```

As the p-value 0.4828 is greater than the .05, we conclude that the smoking habit is independent of the exercise level of the student and hence there is a weak or no correlation between the two variables.

Goodness of Fit test

- A goodness of fit test is a special kind of test of association.
- The **chi-square goodness of fit** test is used to compare the observed distribution to an expected distribution, in a situation where we have two or more categories in a discrete data.
- In other words, it compares multiple observed proportions to expected probabilities.

For example,

> we collected wild tulips and found that 81 were red, 50 were yellow and 27 were white.
>
> Q: Are these colors equally common?

If these colors were equally distributed, the expected proportion would be 1/3 for each of the color.

<u>Syntax</u>

chisq.test(x, p)

- **x**: a numeric vector
- **p**: a vector of probabilities of the same length of x.

Answer to Q1: Are the colors equally common?

```
tulip <- c(81, 50, 27)
res <- chisq.test(tulip, p = c(1/3, 1/3, 1/3))
res
```

```
    Chi-squared test for given probabilities
data:  tulip
X-squared = 27.886, df = 2, p-value = 8.803e-07
```

- The **p-value** of the test is 8.80310^{-7}, which is less than the significance level alpha = 0.05.
- We can conclude that the colors are significantly not commonly distributed with a **p-value** = 8.80310^{-7}.

Types of Dirty Data

(https://www.ringlead.com/blog/the-7-most-common-types-of-dirty-data-and-how-to-clean-them/)

- Duplicate Data
- Outdated Data
- Insecure Data
- Incomplete Data
- Incorrect/Inaccurate Data
- Inconsistent Data
- Too Much Data

1. Duplicate Data

Duplicates are among the worst offenders of data pollution. Duplicates can form in a number of ways including data migration, via data exchanges through integrations, 3rd party connectors, manual entry, and from batch imports. The most common duplicate objects are Contacts, and Accounts.

Polluting your datastore with duplicate data can cause:

- Inflated storage count
- Inefficient workflows and data recovery
- Skewed metrics and analytics

How to clean and prevent duplicates:

Automated solutions are used for detecting and merging duplicates. External solutions to duplicate data allow users to match Leads, Contacts, and Accounts based on customizable criteria and prevent duplicates.

2. Outdated Data

Common causes of outdated data:

- Individuals change roles or companies
- Organizations rebrand or get acquired
- Software and systems evolve past their previous iterations

Such is the nature of the modern digital ecosystem, change is too rapid for the average database to keep up. Organizations need to be able to trust that their data is fresh and up-to-date before using it for insights, decision-making, and analytics.

How to keep data fresh and up to date:

Purging your database of records created before a certain date can help expedite the process of cleaning outdated records.

Data enrichment can also solve the pitfalls of outdated customer records by appending fields with newer information.

3. Insecure Data

Data security & privacy laws are being put into place left and right, giving business extra financial incentive to follow these newly placed laws to a tee.

With steep fines for non-compliance, insecure data is quickly becoming one of the most dangerous types of dirty data.

Digital consent, opt-ins, and privacy notifications are becoming the new norm in an increasingly consumer-centric business landscape.

Major data privacy laws include:

- GDPR in the EU
- California's Consumer Privacy Act (CCPA)
- Maine's Act to Protect the Privacy of Online Consumer Information

Without proper database hygiene, remain within these stringent regulations becomes nearly impossible. For example, let's say an individual does not consent to your data sharing policy but their customer profile is fragmented throughout your organization's various databases. Adhering to this opt-out will need manual intervention to reconcile all instances of this personal information (and let's be real, no one is doing that).

How to remain within data privacy regulations:

Disorderly databases are the most likely candidates to house insecure data. There are several data hygiene practices you can implement to combat insecure data.

- Delete outdated & unusable records
- Merge duplicates to prevent fragmented profiles
- Automate lead-to-account linking
- Consolidate your stack as much as possible

With a clean database, complying with data privacy regulations becomes an afterthought.

4. Incomplete Data

A record can be defined as incomplete if it lacks the key fields you need to process the incoming information before sales and marketing take action.

For example, let’s say your organization is running a campaign to target non-profit institutions. If a new or existing record is missing the ‘industry’ or ‘sector’ fields, it will not be included in smart lists for the campaign and a valuable revenue opportunity may be missed.

How to Fix Incomplete Data:

Enriching your data and to automate the filling of empty fields and gain a more complete profile of targets and customers.

5. Inaccurate/Incorrect Data

Incorrect data is data that is stored in the improper location e.g a text field containing a numerical value

Inaccurate data, on the other hand, occurs when a field is filled but the information is not correct e.g a fake email address

These each can cause a slew of issues, including poor targeting and segmentation, irrelevant non-personalized messaging, mistimed or nonexistent email delivery, and a lack of competitive insights.

How to clean incorrect and inaccurate data:

Keeping track of all data entry points and diagnosing the cause of inaccurate data is the first step towards combating this type of bad data. If the problem is caused by external data sources (web forms or connected systems), seeking an external solution is the best option to maintain accuracy.

6. Inconsistent Data

Just like how duplicate records exist in various places within your database, multiple versions of the same data elements can exist across different records.

Inconsistent (non-standardized) data is data that looks different but represents the same thing.

For example, let’s say you were targeting decision makers for an upcoming email blast and wanted to segment all “Vice President” roles into one person. ‘V.P’ ‘v.p’ ‘VP’ & ‘Vice Pres’ all mean the same thing, yet will only be included if you are certain all variations exist in your smart list or campaign. Inconsistent data hurts analytics and make segmentation that much more difficult when you have to account for all variables of the same title, industry, or other criteria.

How to standardize your data:

First, create standard naming conventions and ensure your organization follows this closely going forward. As for existing inconsistent records, tools can normalize records in batch for more unified field names and more accurate segmentation.

Integrating in a data management tool that can standardize data from multiple sources can help to create a centralized approach to data management. This will enable data to be processed, analyzed,

and leveraged across each department within an organization, enabling a successful data sharing strategy and increased accessibility throughout your organization.

7. Too Much Data

Yes, **data hoarding** is a thing.

Data Hoarding causes:

- slower data exchange
- inflated record counts
- failure to stay within storage compliance limits.

Maintaining a sleek (but not small) database is a big part of data hygiene, driving alignment between departments and improving accessibility throughout your organization.

How to reduce database size:

While it might seem like “too much data” can never be a bad thing, more often than not, a good portion of the data simply isn’t usable. This means that your team is spending excess time digging through the bad so they can get to the good. Data hoarding and outdated data go hand in hand. So you will find that these two types of dirty data can be solved at the same time.